

TÀI LIỆU KHỞI ĐỘNG DỰ ÁN

Tên tạm thời: Security Context Engine / Security Hunt Harness

1. TẦM NHÌN

Xây dựng một security harness:

- Tích hợp vào Claude Code, Codex và các coding agent dưới dạng Agent Skill.
- Không phụ thuộc một model hoặc provider duy nhất.
- Hỗ trợ whitebox, blackbox và kết hợp cả hai.
- Phân rã codebase/hệ thống thành nhiều vùng nhỏ để agent khám phá.
- Lưu lại kiến thức, evidence, coverage và kết quả giữa các lần chạy.
- Chỉ phân tích lại vùng thay đổi và vùng bị ảnh hưởng.
- Có thể hiểu mối quan hệ giữa nhiều website/service nội bộ.
- Kết hợp AI với SAST, DAST, SCA và runtime testing.
- Finding quan trọng phải có evidence/PoC tái hiện được.
- Tối ưu theo chi phí trên mỗi finding được xác nhận, không theo số alert.

Định vị:

Không chỉ là “AI scanner” hoặc “skill OWASP”.

Đây là một engine duy trì System Security Context, sau đó cấp đúng lát cắt context cho các hunter/validator agent.

2. LỢI THẾ CẠNH TRANH MỤC TIÊU

Không cạnh tranh chỉ bằng:

- Prompt hay hơn.
- Spawn nhiều agent hơn.
- Hỗ trợ nhiều model hơn.
- Có checklist OWASP.
- Tích hợp Semgrep/Trivy/ZAP.

Các phần trên sẽ nhanh chóng trở thành commodity.

Lợi thế nên nằm ở:

1. Persistent system context:
 - Không phải khám phá lại từ đầu mỗi lần hunt.
2. Cross-system understanding:
 - Source code.

- Runtime behavior.
- Authentication/identity.
- Website/service relationships.
- Shared libraries và dependencies.

3. Incremental semantic update:

- Code thay đổi.
- Xác định fact/path/finding/consumer nào bị ảnh hưởng.
- Chỉ reanalyze/revalidate phần cần thiết.

4. Cost-aware scheduling:

- Deterministic tools trước.
- Model rẻ cho fact extraction.
- Model mạnh cho cross-region reasoning.
- Model độc lập cho validation.

5. Evidence-first findings:

- Không report suy đoán như vulnerability thật.
- Có control case, proof case, impact và proof gap.

3. NGUYÊN TẮC KIẾN TRÚC

3.1. Model là stateless compute

Không coi context window của model là bộ nhớ.

Mọi dữ liệu quan trọng phải lưu bên ngoài:

- Architecture.
- Threat model.
- Security invariants.
- Facts.
- Assumptions.
- Hypotheses.
- Findings.
- Coverage.
- Evidence.
- Previous runs.
- Validation attempts.
- Dismissal reasons.
- Tool results.

3.2. Deterministic orchestration

Code thông thường chịu trách nhiệm:

- Queue.
- Retry.
- Timeout.
- Budget.
- Scope.
- Cache.
- Deduplication.
- State transition.
- Concurrency.
- Rate limit.
- Approval gate.
- Incremental invalidation.

LLM chịu trách nhiệm:

- Hiểu semantics.
- Nối dữ kiện.
- Đề xuất hypothesis.
- Thiết kế validation.
- Phản biện finding.
- Giải thích impact.
- Đề xuất remediation.

3.3. Discovery và validation độc lập

Không để agent tìm finding tự xác nhận finding đó.

Pipeline:

Recon

- Hunt
- Candidate hypothesis
- Independent adversarial validator
- Runtime/static proof
- Confirmed finding
- Human acceptance
- Minimal patch
- Revalidate fix

Có thể dùng model khác nhau cho discovery và validation để giảm correlated error.

3.4. Context retrieval theo nhu cầu

Không dump toàn bộ repository hoặc graph vào prompt.

Thứ tự retrieval:

- Exact route/symbol.

- Callers/callees.
- Graph neighborhood.
- Source-to-sink hoặc constraint slice.
- Auth/authz context.
- Related configuration.
- Runtime observations.
- Lexical search.
- Vector similarity.

3.5. Whitebox và blackbox dùng chung evidence model

Whitebox sinh hypothesis.

Blackbox/runtime cố chứng minh hoặc bác bỏ hypothesis.

Ví dụ:

Source analysis:

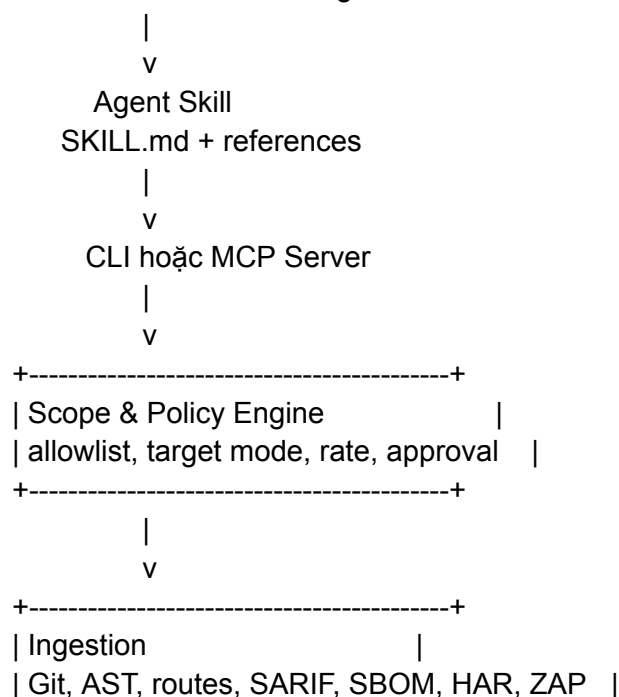
POST /invoice/:id có thể thiếu tenant ownership check.

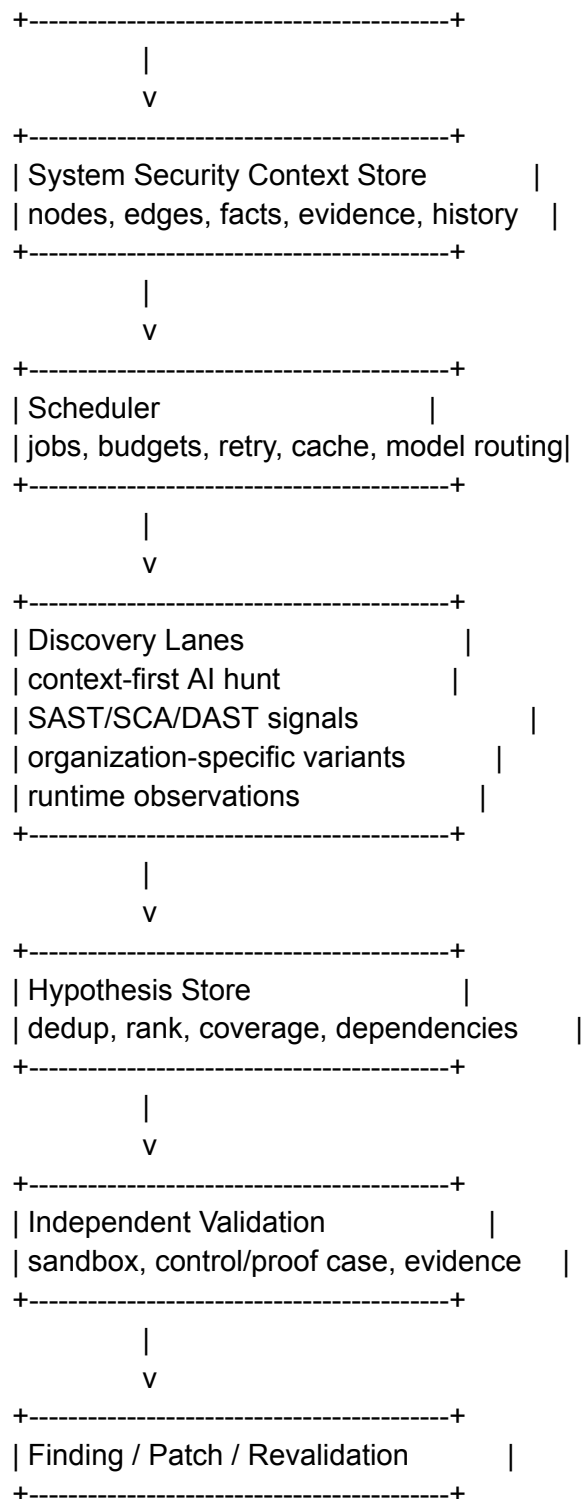
Runtime validation:

- Control: tenant A đọc invoice của tenant A $\rightarrow$ 200.
- Proof: tenant A đọc invoice của tenant B $\rightarrow$ 200.
- Impact: lộ dữ liệu tenant B.

4. KIẾN TRÚC TỔNG THỂ

Claude Code / Codex / Agent khác





=====

5. SYSTEM SECURITY CONTEXT

=====

5.1. Các loại node

Whitebox:

- Repository.
- Commit.
- Service.
- Module/package.
- File.
- Class/function/symbol.
- Route/endpoint.
- Middleware.
- Source.
- Validator/sanitizer.
- Sink.
- Authentication check.
- Authorization check.
- Role/permission.
- Database/table/model.
- Queue/topic.
- External API.
- File/object storage.
- Dependency.
- Config/secret.
- Finding.
- Evidence.
- Test case.

Blackbox/runtime:

- Domain/subdomain.
- Website/application.
- Page.
- API endpoint.
- HTTP method.
- Parameter/header/cookie.
- Authentication state.
- User role.
- Business object.
- Workflow state.
- Redirect.
- WebSocket/GraphQL operation.
- Request/response observation.
- Browser action.
- DAST alert.

5.2. Các loại edge

- CALLS
- IMPORTS
- EXPOSES
- HANDLED_BY

- READS_FROM
- WRITES_TO
- VALIDATES_WITH
- SANITIZES_WITH
- AUTHENTICATES_WITH
- AUTHORIZED_BY
- SHARES_IDENTITY_WITH
- SHARES_SESSION_WITH
- ISSUES_TOKEN_FOR
- REDIRECTS_TO
- TRUSTS
- PRODUCES
- CONSUMES
- DEPLOYS_AS
- REACHES
- AFFECTS
- EVIDENCED_BY
- DERIVED_FROM

5.3. Provenance bắt buộc

Mọi fact cần có:

- Origin.
- Evidence references.
- Source revision.
- Extractor/model version.
- Confidence.
- Created time.
- Stale status.

Không có provenance thì chỉ được coi là assumption hoặc hypothesis.

=====

6. CANONICAL DATA MODEL

=====

Phân biệt rõ:

OBSERVATION:

Một hành vi được quan sát.

Ví dụ: GET /users/123 trả HTTP 200.

FACT:

Dữ kiện có provenance.

Ví dụ: route trên được xử lý bởi UserController.show.

ASSUMPTION:

Điều chưa xác minh hoặc do con người khai báo.
Ví dụ: service này chỉ được truy cập từ mạng nội bộ.

INVARIANT:

Thuộc tính bảo mật phải luôn đúng.
Ví dụ: tenant không được đọc object của tenant khác.

HYPOTHESIS:

Vulnerability candidate chưa xác nhận.
Ví dụ: endpoint có thể thiếu object ownership check.

FINDING:

Hypothesis đã đạt ngưỡng evidence.
Ví dụ: tenant A thực tế đã đọc được invoice của tenant B.

EVIDENCE:

Source location, request/response, log, trace, screenshot, test result hoặc artifact chứng minh.

=====

7. FINDING STATE MACHINE

=====

SIGNAL

→ HYPOTHESIS
→ UNDER_REVIEW
→ VALIDATION_PLANNED
→ VALIDATION_BLOCKED
→ REJECTED
→ PARTIALLY_VALIDATED
→ CONFIRMED
→ ACCEPTED
→ PATCH_PROPOSED
→ FIX_VERIFIED
→ CLOSED

Cho phép:

CLOSED → REOPENED

khi:

- Code thay đổi.
- Dependency thay đổi.
- Threat-model assumption thay đổi.
- Runtime environment thay đổi.
- Có bypass mới.
- Evidence cũ không còn hợp lệ.

8. THREAT MODEL VÀ SECURITY INVARIANTS

Không bắt đầu chỉ từ OWASP checklist.

Mỗi target cần threat model gồm:

- System purpose.
- Assets.
- Entry points.
- Untrusted inputs.
- Attacker identities.
- Trust boundaries.
- Authentication assumptions.
- Authorization model.
- Sensitive operations.
- Sensitive data paths.
- Deployment assumptions.
- Security invariants.
- Areas cần ưu tiên.
- Exclusions.

Threat model phải editable.

Phân biệt:

- AI-generated fact.
- Human-declared fact.
- Runtime-observed fact.

Có thể hỗ trợ:

repo/SECURITY.md
repo/services/payment/SECURITY.md
repo/services/payment/refund/SECURITY.md

File gần component hơn có thể bổ sung context cụ thể, nhưng engine phải đối chiếu với code/runtime để phát hiện tài liệu cũ.

9. WHITEBOX PIPELINE

1. Inventory:

- Language/framework.
- Entry points.

- Routes.
- Middleware.
- Auth/authz.
- Database.
- Serialization.
- File/network operations.
- Config/dependencies.

2. Deterministic extraction:

- AST.
- Symbols.
- Call/import graph.
- Route graph.
- Tool signals.

3. Semantic partition:

- Route → controller → service → repository.
- Domain/module.
- Shared middleware.
- Data-flow paths.
- Deployment/service boundary.

4. Leaf fact extraction:

- Sources.
- Sinks.
- Validation.
- Sanitization.
- Auth checks.
- Side effects.
- Unknowns.
- Suspicious paths.

5. Cross-region synthesis:

- Nối paths.
- Kiểm tra invariant.
- Tìm missing/broken controls.
- Tạo hypothesis.

6. Rank và schedule validation.

7. Lưu toàn bộ context để tái sử dụng.

Không chia code chỉ theo folder. Folder chỉ là seed; cần xem call graph, routes, data models và shared security controls.

=====

10. BLACKBOX PIPELINE

=====

Nguồn dữ liệu:

- OpenAPI.
- Postman collection.
- HAR.
- Browser session.
- ZAP spider/passive scan.
- GraphQL schema.
- Runtime logs/traces nếu có quyền.
- Service catalog.

Blackbox graph tập trung vào:

- Role matrix.
- Session/token lifecycle.
- Object ownership.
- Workflow state.
- Cross-domain cookies.
- Redirects.
- API object relationships.
- Behavior differences giữa role/user.

Mode target:

PRIMARY TARGET:

- Deep discovery.
- Active testing nếu được phép.

RELATED TARGET:

- Ghi nhận dependency/identity/redirect.
- Passive-only mặc định.
- Không active scan nếu chưa được approve.

Ví dụ policy:

primary_targets:

- app-a.internal

related_targets:

discovery: passive-only
active_testing: false

cross_domain:

record_relationships: true
follow_redirects: same-scope-only

=====

11. DISCOVERY LANES

Không để SAST findings bias toàn bộ AI hunt.

Chạy các lane tương đối độc lập:

Lane A — Context-first AI hunt:

- Repository.
- Threat model.
- Invariants.
- Architecture.
- Không đưa toàn bộ SAST alerts ngay từ đầu.

Lane B — Deterministic tool signals:

- Semgrep.
- CodeQL.
- Trivy.
- Secret scanning.
- ZAP.
- Dependency/SBOM.

Lane C — Historical/organization knowledge:

- Lỗi team thường gặp.
- Dismissal reasons.
- Known unsafe wrappers.
- Previous incidents.
- Known safe controls.

Lane D — Runtime observations:

- HAR.
- Browser/API behavior.
- Logs/traces.
- Authentication roles.

Sau đó merge vào unified hypothesis store.

12. AGENT HIERARCHY

Tier 0 — Không dùng LLM:

- Hash/diff.
- AST.
- Route extraction.
- Tool execution.
- Cache.

- Dedup.
- Scheduling.
- Scope enforcement.

Tier 1 — Model rẻ:

- Fact extraction.
- Region summary.
- Source/sink/sanitizer classification.
- Side-effect identification.
- Structured JSON output.

Tier 2 — Model trung bình:

- Cross-region linking.
- Authorization reasoning.
- Constraint transformation.
- Tool finding triage.
- Validation planning.

Tier 3 — Model mạnh:

- Cross-service/system reasoning.
- Business logic.
- Contradictory evidence.
- High-impact hypotheses.
- Complex validation.
- Final report review.

Discovery và validation ưu tiên dùng model/agent khác nhau.

=====

13. COVERAGE MODEL

=====

Coverage không chỉ là “repo đã scan”.

Dùng ma trận:

area × attack class

Ví dụ:

Area: payment API

- Access control: deep
- Injection: standard
- Auth: deep
- Crypto: partial

- Business logic: partial

Mỗi coverage record có:

- Reviewed entities.
- Depth.
- Attack class.
- Evidence.
- Deferred surfaces.
- Proof gaps.
- Model/tool version.
- Source revision.
- Stale status.

Gapfill job tìm những ô còn thiếu và tạo task mới.

Kết luận scan phải viết:

“Không có finding được xác nhận trong phạm vi và coverage này.”

Không viết:

“Hệ thống không có vulnerability.”

=====

14. VALIDATION VÀ POC

=====

Finding chính thức cần:

- Target/revision.
- Attacker identity.
- Preconditions.
- Trust boundary.
- Security invariant bị phá.
- Entry point.
- Attack path.
- Control case.
- Proof case.
- Observed result.
- Impact thực tế.
- Evidence artifacts.
- Confidence.
- Limitations.
- Root cause.
- Remediation.
- Fix verification procedure.

Validation phải ưu tiên:

- Original untouched source.
- Isolated environment.
- Test accounts.
- Test objects.
- Non-destructive operations.
- Rate limits.
- Redaction secrets/PII.

Nếu không validate được:

- Không hallucinate.
- Ghi VALIDATION_BLOCKED.
- Tạo wishlist item.

Wishlist ví dụ:

- Cần build environment.
- Cần database fixture.
- Cần test account role admin.
- Cần production-like config.
- Cần service dependency.
- Cần browser environment.

Sau khi môi trường được cung cấp, resume đúng task cũ.

=====

15. REMEDIATION

=====

Một finding, một remediation task.

Quy trình:

1. Reproduce finding.
2. Xác định root cause.
3. Tạo minimal patch.
4. Thêm regression test.
5. Rerun original proof.
6. Kiểm tra legitimate behavior.
7. Kiểm tra nearby bypass.
8. Chạy relevant repository tests.
9. Human review.
10. Đóng finding có chủ đích.

Không bulk fix tất cả finding trong một context/session.

=====

16. TOOL INTEGRATION

=====

MVP:

- Semgrep: pattern SAST.
- Trivy: dependency/container/laC/secret/SBOM.
- OWASP ZAP: DAST.
- Playwright: browser automation.
- SARIF: finding interchange.
- CycloneDX: SBOM.

Giai đoạn sau:

- CodeQL: deeper data flow.
- Joern/CPG: cross-language code graph.
- Gitleaks nếu cần.
- OpenTelemetry/service traces.
- Fuzzing/micro-fuzzer.
- SMT solver cho constraint reasoning.

Tool alert chỉ là SIGNAL, không phải CONFIRMED finding.

=====

17. MODEL-INDEPENDENT INTERFACE

=====

Model provider interface:

- generate_structured()
- generate_reasoned()
- context_limit()
- supports_tools()
- supports_local()
- estimated_cost()
- model_capabilities()

Không hardcode model name vào workflow.

Routing theo capability:

fact-extraction:

tier: cheap
structured-output: required

hypothesis-linking:

tier: medium

cross-system-reasoning:
tier: strong

validation-review:
tier: independent

Log:

- Provider/model.
- Prompt/schema version.
- Input/output tokens.
- Cost.
- Latency.
- Cache hit.
- Task type.
- Outcome usefulness.

=====

18. TỐI ƯU TOKEN VÀ CHI PHÍ

=====

Không chỉ bắt AI “nói ít”.

Áp dụng:

1. Structured output thay vì prose.
2. Progressive disclosure trong Agent Skill.
3. Graph retrieval trước vector search.
4. Chỉ lấy code slice cần thiết.
5. Cache theo content hash.
6. Dedup hypothesis trước validation.
7. Model rẻ cho extraction.
8. Model mạnh chỉ khi risk/uncertainty cao.
9. Previous findings để tránh khám phá lại.
10. Coverage gap để tránh scan trùng.
11. Summary có provenance và expiry.
12. Context request phải có lý do.

Task priority:

priority =
expected_risk
× expected_information_gain
× coverage_gap
÷ estimated_cost

=====

19. INCREMENTAL UPDATE

=====

Khi code thay đổi:

1. Xác định artifact thay đổi.
2. Cập nhật AST/symbol/route/data-flow liên quan.
3. Tìm facts phụ thuộc artifact.
4. Mark stale summaries/hypotheses.
5. Tìm callers, consumers và shared services bị ảnh hưởng.
6. Reopen findings cần revalidate.
7. Mark coverage stale.
8. Chỉ schedule impact closure.

Không chỉ dựa vào file hash.

Phải xem:

- Content change.
- Dependency edge change.
- Config change.
- Route/middleware change.
- Rule version.
- Prompt/model version.
- Runtime assumption change.

=====

20. STORAGE MVP

=====

Không cần Neo4j ngay.

Dùng:

- SQLite ban đầu hoặc PostgreSQL.
- Object/file storage cho artifacts.
- Full-text search.
- Vector search chỉ thêm khi có use case rõ.

Tables:

- projects
- repositories
- snapshots
- artifacts
- nodes
- edges
- facts

- assumptions
- invariants
- observations
- hypotheses
- findings
- evidence
- coverage
- tasks
- agent_runs
- validation_attempts
- patches
- rules
- scope_policies

21. AGENT SKILL

Skill chỉ là thin adapter.

Cấu trúc:

```
skills/security-hunt/
  SKILL.md
  references/
    workflow.md
    threat-model.md
    attack-classes.md
    validation.md
    reporting.md
    safety-policy.md
  schemas/
  scripts/
```

Skill chịu trách nhiệm:

- Parse yêu cầu user.
- Xác định scan mode.
- Xác nhận scope.
- Gọi CLI/MCP.
- Hướng dẫn review output.
- Yêu cầu approval nếu cần.

Core engine không nằm trong SKILL.md.

22. CÁC HỆ THỐNG CẦN THAM KHẢO

Cloudflare Security Audit Skill:

<https://github.com/cloudflare/security-audit-skill>

Cloudflare Vulnerability Harness:

<https://blog.cloudflare.com/build-your-own-vulnerability-harness/>

Học:

- Recon/Hunt/Validate.
- Adversarial validator.
- Structured output.
- Coverage.
- Gapfill.
- Dedup.
- Cross-repo tracing.
- External state.
- Model diversity.

Codex Security:

<https://developers.openai.com/codex/security>

Codex Security research preview:

<https://openai.com/index/codex-security-now-in-research-preview/>

Threat model:

<https://developers.openai.com/codex/security/threat-model>

Scan workflow:

<https://developers.openai.com/codex/security/plugin/scans>

Deep scan:

<https://developers.openai.com/codex/security/plugin/deep-scans>

Fix/verify:

<https://developers.openai.com/codex/security/plugin/fix-findings>

Học:

- Editable threat model.
- Security invariants.
- Standard/deep/diff mode.
- Validation sandbox.
- Proof gap.
- Sealed scan bundle.
- One-finding remediation.
- Revalidation.

Vulnhuntr:

<https://github.com/protectai/vulnhuntr>

Học:

- Iterative symbol/context retrieval.
- Call-chain construction.
- Agent yêu cầu thêm context theo nhu cầu.

Shannon:

<https://github.com/KeygraphHQ/shannon>

Học:

- Whitebox-guided runtime testing.
- Browser/API proof.
- Authenticated workflows.
- Only report reproducible findings.
- Resumable workspace.

GitHub Security Lab Taskflow:

<https://github.blog/security/ai-supported-vulnerability-triage-with-the-github-security-lab-taskflow-agent/>

Học:

- Task decomposition.
- Fresh context.
- Intermediate notes.
- LLM + deterministic MCP tools.
- Reusable dismissal reasons.

AURI:

<https://www.endorlabs.com/platform>

Học:

- Code context graph.
- Code/dependency/container relationships.
- Agent dùng graph query thay vì graph dump.
- Reachability/exploitability evidence.

XBOW:

<https://xbow.com/platform>

Học:

- Proof-first blackbox testing.
- Portfolio orchestration.
- Model routing.
- Governance.
- Non-destructive defaults.

Buttercup:

<https://trailofbits.com/buttercup/>

Học:

- Static analysis + AI-guided fuzzing.
- Multi-agent discovery/patch.
- Prove bug và verify fix.
- Native-code pipeline.

OWASP Top 10:

<https://owasp.org/Top10/2025/>

OWASP ASVS:

<https://owasp.org/www-project-application-security-verification-standard/>

OWASP WSTG:

<https://owasp.org/www-project-web-security-testing-guide/>

Agent Skills specification:

<https://agentskills.io/specification>

=====

23. COPY/REIMPLEMENT STRATEGY

=====

Nguyên tắc:

- Copy design pattern.
- Fork khi license phù hợp.
- Tự sở hữu core contracts.
- Không ghép trực tiếp mọi repository.

Cloudflare skill:

- License MIT.
- Có thể fork/modify.
- Giữ copyright/license notice.
- Dùng làm baseline rồi refactor.

Shannon:

- AGPL-3.0.
- Cần thận nếu sản phẩm đóng/SaaS.
- Ưu tiên học pattern và tự implement.
- Commercial license có thể cần nếu dùng trực tiếp.

Vulnhuntr:

- AGPL-3.0.
- Ưu tiên tự implement context retrieval.

Buttercup:

- AGPL-3.0.

- Dùng làm tài liệu kiến trúc hoặc external experiment.
- Không nhúng vào proprietary core nếu chưa review license.

Codex Security/AURI/XBOW:

- Closed/proprietary.
- Học workflow công khai.
- Không có source để copy.
- Tự implement từ functional specification.

Không:

- Đưa nguyên code AGPL vào AI rồi yêu cầu rewrite để né license.
- Copy nguyên schema/prompt/docs đặc thù mà không kiểm tra license.
- Port source sang ngôn ngữ khác rồi coi là code mới.
- Dùng benchmark marketing làm chứng minh cho sản phẩm.

Nên:

- Viết functional specification riêng.
- Lưu design docs và commit history.
- Tự viết schema/test.
- Review license trước khi thương mại hóa.
- Nhờ chuyên gia pháp lý kiểm tra nếu sản phẩm có giá trị lớn.

24. EVALUATION HARNESS

So sánh:

1. Tool-only:
 - Semgrep/CodeQL/Trivy/ZAP.
2. Single-agent:
 - Một model đọc repository.
3. Cloudflare skill nguyên bản.
4. Multi-agent không persistent memory.
5. Hệ thống của mình:
 - Persistent context.
 - Coverage.
 - Independent validation.
 - Incremental update.

Metrics:

- Recall trên seeded/known vulnerabilities.
- Precision.
- Confirmed findings / signals.
- False-positive rate.
- Dev acceptance rate.
- Time to first confirmed finding.
- Total scan cost.
- Cost per confirmed finding.
- Token per confirmed finding.
- PoC reproduction success.
- Duplicate finding rate.
- Coverage.
- Incremental rescan savings.
- Fix verification success.
- Variability giữa nhiều run.

Metric sản phẩm chính:

Chi phí và thời gian trên mỗi finding được dev chấp nhận.

=====

25. ROADMAP

=====

PHASE 0 — Baseline research, 1–2 tuần

- Fork/chạy Cloudflare skill nguyên bản.
- Chọn 5–10 benchmark repositories.
- Chạy nhiều lần để đo variability.
- Thử Codex Security nếu có quyền truy cập.
- Thử Shannon/Vulnhuntr trong môi trường nghiên cứu.
- Lập capability/license matrix.
- Chốt canonical schemas.

Deliverables:

- Benchmark corpus.
- Baseline report.
- Architecture decision record.
- Finding/evidence/task schemas.
- Threat model schema.

PHASE 1 — Skill + persistent harness, 3–5 tuần

- Agent Skill.
- CLI/MCP.
- SQLite.
- Recon/Hunt/Validate pipeline.

- External task state.
- Resume/retry.
- Model provider interface.
- Structured output.
- Coverage matrix.
- Cost tracking.

PHASE 2 — Whitebox MVP, 4–6 tuần

Chọn một stack:

- TypeScript/Node.js hoặc
- Java/Spring.

Tính năng:

- AST/symbol/routes.
- Semantic regions.
- Sengrep.
- Trivy.
- Fact extraction.
- On-demand context retrieval.
- Cross-region hypothesis.
- Incremental cache.

Deep hunt:

- Broken access control.
- Authentication.
- Injection.
- Security misconfiguration.
- Supply-chain failures.

PHASE 3 — Runtime validation, 3–5 tuần

- Isolated environment.
- App launch recipe.
- Test accounts.
- Browser/API executor.
- Control/proof case.
- Evidence capture.
- Validation wishlist.
- Original-source verification.
- Report generation.

PHASE 4 — Remediation, 2–4 tuần

- Finding acceptance.

- Minimal patch.
- Regression test.
- Reproducer rerun.
- Nearby bypass checks.
- Fix verification.
- SARIF/JSON/Markdown export.

PHASE 5 — Cross-repo/system context, 4–8 tuần

- Dependency/consumer graph.
- Shared auth/identity.
- Cross-domain redirects/cookies.
- Service-to-service trust.
- Multi-web coverage.
- Primary-target/passive-related-target mode.
- Cross-system hypotheses.

PHASE 6 — Organization learning

- Confirmed finding → pattern candidate.
- Human-reviewed custom rules.
- Positive/negative fixtures.
- Dismissal reasons.
- Known safe/unsafe helpers.
- Recurrence detection.
- CI integration.

=====

26. MVP DEFINITION OF DONE

=====

MVP hoàn thành khi:

- Cài được như Agent Skill.
- Chạy được từ Claude Code và Codex qua cùng core CLI/MCP.
- Không hardcode một model.
- Scan được một repository thuộc stack đã chọn.
- Xây được route/function/auth/source/sink context.
- Có persistent state và resume.
- Có coverage matrix.
- Có ít nhất ba specialist hunter.
- Discovery và validation độc lập.
- Tool output được normalize.
- Finding có provenance.
- Có signal/hypothesis/confirmed states.
- Finding confirmed có control/proof case hoặc evidence tương đương.
- Đo được tokens, cost, latency.
- Incremental scan rẻ hơn full scan.

- Report dev có thể hiểu và tái hiện.
- Scope và approval policy hoạt động.

=====

27. CẤU TRÚC REPOSITORY ĐỀ XUẤT

=====

security-context-engine/

skills/

security-hunt/

SKILL.md

references/

schemas/

scripts/

core/

scheduler/

state/

scope/

policy/

graph/

artifacts/

cache/

findings/

evidence/

coverage/

incremental/

adapters/

git/

semgrep/

trivy/

codeql/

zap/

playwright/

sarif/

cyclonedx/

analyzers/

whitebox/

blackbox/

partitioning/

impact-analysis/

agents/

recon/

fact-extractor/

access-control-hunter/

auth-hunter/
injection-hunter/
business-logic-hunter/
cross-service-hunter/
validator/
reporter/
remediation/

providers/
openai/
anthropic/
local/
router/

schemas/
artifact.schema.json
fact.schema.json
task.schema.json
threat-model.schema.json
finding.schema.json
evidence.schema.json
coverage.schema.json
validation.schema.json

policies/
benchmarks/
fixtures/
reports/
docs/

=====

28. VIỆC NÊN LÀM NGAY

=====

1. Fork Cloudflare security-audit-skill để làm baseline.
2. Không sửa ngay; chạy nguyên bản trên benchmark.
3. Tạo repository core riêng.
4. Viết canonical schemas trước.
5. Viết SQLite task/artifact store.
6. Bọc skill bằng CLI/MCP.
7. Externalize Recon/Hunt/Validate output.
8. Thêm adversarial validator.
9. Thêm coverage matrix.
10. Đo token/cost/findings qua nhiều run.
11. Sau đó mới thêm graph và incremental update.
12. Cuối cùng mới mở rộng sang runtime và multi-web.

Quy tắc phát triển:

Mỗi capability mới phải chứng minh ít nhất một trong các điều sau:

- Tăng confirmed findings.
- Giảm false positives.
- Tăng coverage.
- Giảm cost.
- Giảm runtime.
- Tăng PoC success.
- Tăng dev acceptance.
- Giảm incremental rescan cost.

Nếu không cải thiện metric nào, không giữ capability đó.

=====

29. KẾT LUẬN

=====

Chiến lược phù hợp:

Fork để học nhanh

- đo baseline
- trích design pattern
- chuẩn hóa contract
- xây core riêng
- thay dần các phần fork
- giữ external tools dưới dạng adapter.

Không xây lại mọi thứ từ số 0.

Không ghép trực tiếp mọi codebase thành một sản phẩm.

Core cần tự sở hữu:

- Data model.
- State machine.
- Scheduler.
- System context.
- Incremental invalidation.
- Scope/policy.
- Evaluation harness.

Mục tiêu cuối:

Một model-agnostic, cost-aware, persistent và incremental security harness có khả năng hiểu code, runtime, identity và quan hệ giữa nhiều website/service; cung cấp đúng context cho hunter và chỉ đưa finding đến dev khi có evidence đủ mạnh.